

An On-Premise, Organization-Bound LLM Assistant for Context-Aware Software Engineering

Preet Patel

Abstract

This project proposes to develop a local Large Language Model (LLM) assistant tailored for software engineering tasks within a company. Unlike cloud-based assistants (e.g., ChatGPT or GitHub Copilot) that lack access to private codebases, our system will reside on-premises and integrate with the organization’s internal knowledge. We will combine a local LLM with Retrieval-Augmented Generation (RAG) techniques, including knowledge graph-augmented retrieval (GraphRAG) and standardized tool interfaces (MCP), to enable context-aware question answering. Over an 8-week timeline, we plan to build a prototype that can answer developer queries using proprietary code, documentation, and logs, all while ensuring data never leaves the company. The outcome will be a proof-of-concept assistant demonstrating improved relevance on internal queries that generic LLMs cannot handle.

1 Motivation and Problem

Recent advances in LLMs like OpenAI’s GPT-3 and GPT-4 have shown that AI can assist developers by generating code and answering programming questions. Tools such as GitHub Copilot, powered by OpenAI Codex (a GPT-3 model fine-tuned on public GitHub code), illustrate the potential of AI pair programmers. However, these generic models are trained on broad public data and lack knowledge of any specific organization’s codebase, architecture, and internal tools. They cannot access confidential repositories or up-to-date internal documentation by default. This leads to a gap: developers often have questions that off-the-shelf LLMs simply cannot answer because the necessary context is private or organization-specific. For example, a developer might ask:

- “Which microservice owns the API endpoint `‘/data/export’`, and where is it implemented in our codebase?”
- “Why did our nightly build fail with error code 137, and how can it be fixed given our CI/CD setup?”

- “Where in our internal wiki is the usage guideline for the custom library ‘AuthNService’?”

Generic LLMs have no knowledge of the above, as the answers lie in company-specific resources (source code, build logs, internal documentation, etc.). Additionally, privacy and IP concerns often prevent using external AI services on proprietary code. This project is motivated by the need for an AI assistant that operates securely on-premises and is “organization-bound” - i.e., aware of the company’s own code and knowledge. By doing so, we aim to dramatically improve developer productivity for in-house projects, filling the void left by current LLMs’ inability to handle private context.

2 Proposed Solution

We propose an architecture that tightly integrates a local LLM with the organization’s knowledge sources through a multi-layer approach. The solution has three main layers:

1. Local LLM Core - At the heart is a locally deployed LLM (such as Code LLaMA or a fine-tuned variant) that will generate answers and code suggestions. Running on-premise ensures that proprietary data and queries are processed securely, without sending information to external APIs. While the base model will possess general coding and natural language capabilities (comparable to GPT-style models), it will be augmented at runtime with organization-specific context rather than retrained on private data from scratch.

2. Knowledge Retrieval Layer - To provide the LLM with relevant context, we will implement Retrieval-Augmented Generation. All available company data - code repositories, design docs, wiki pages, ticket logs, etc. - will be indexed for search. Given a developer’s query, the system will retrieve pertinent fragments (e.g. code snippets, API docs, configuration files) and supply them to the LLM’s prompt. This retrieval layer will combine a vector search over embeddings (to find semantically relevant text) with structured retrieval using a knowledge graph. In particular, we plan to leverage GraphRAG, which builds a knowledge graph from the textual corpus and uses it to enhance query answering. By linking entities such as services, functions, and configuration parameters, the assistant can perform multi-hop reasoning (e.g. tracing how an error propagates through different modules). This structured approach has been shown by Microsoft to substantially improve answering complex, connected questions over baseline RAG. The retrieved context (whether code snippets or summarized knowledge subgraphs) will be appended to the LLM prompt to ground the assistant’s answer in real internal data.

3. Integration and Interface Layer - On top of the LLM and retrieval components, we will build an interface layer that connects the assistant with the developer’s environment and tools. We intend to follow the principles of the recently introduced Model Context Protocol (MCP) by Anthropic, which provides a standardized way for AI assistants to interface with external systems.

Using an MCP-style interface, our assistant will securely interact with internal services. For example, instead of naively searching the entire codebase (which could be slow for large monorepos), the assistant could invoke a code search tool or query the version control system through a predefined MCP connector. MCP offers a unified protocol to access data sources and tools, replacing ad-hoc integrations with a consistent API. In our design, we will implement a few lightweight MCP-like connectors (or adapt existing ones) for key resources: e.g., a code search server, a documentation database, and perhaps a build status API. This allows the LLM to ask for information in a structured way (like “search for function X in repository Y”) and get a reliable, formatted result, rather than hallucinating or relying on brittle prompt engineering. The interface layer will manage these tool calls and feed results into the retrieval/LLM pipeline. We will expose the assistant to users either via a CLI chat interface or a simple web/UI chat, enabling developers to ask questions conversationally and receive answers with citations into internal docs or links to code.

Overall, the proposed system tightly couples an LLM with the organization’s knowledge sources and tools. By designing the solution in layers (LLM, retrieval, integration), we ensure modularity and focus: the LLM handles language and reasoning; the retrieval layer handles finding the right context; and the interface layer handles safe interaction with existing systems. This approach is expected to yield an assistant that can answer questions about internal code and processes that were previously unanswerable by standalone LLMs.

3 Use of GraphRAG and MCP-Style Interfaces

GraphRAG for Context-Awareness: A key innovation in our approach is the use of GraphRAG (Graphs + RAG) to organize and retrieve context. Standard RAG systems typically retrieve a set of top- k text chunks via semantic similarity, which can fail on complex queries that require joining information from multiple sources. GraphRAG addresses this by extracting a knowledge graph from the corpus: nodes represent important entities (e.g. components, functions, configurations) and edges represent relations (calls, ownership, dependencies, etc.). We will apply this to our company’s data (for instance, auto-generating a graph of services and their API endpoints, or data models and their relationships). When a question is asked, instead of relying purely on keyword or embedding search, the system can traverse the knowledge graph to gather context. For example, for a question about an API endpoint, GraphRAG could help identify the owning service, related modules, and recent incidents involving it by following edges in the graph. These graph-derived context snippets (along with natural language summaries of relevant nodes/communities) will be provided to the LLM. By leveraging GraphRAG, we expect the assistant to handle “multi-hop” questions and synthesizing insights across documents more effectively than a flat retrieval approach. This will make the assistant truly context-aware about the structure of the software system and not just a text lookup tool.

MCP-Style Tool Interfaces: In tandem, we will incorporate the Model Context Protocol design to interface with tools in a robust manner. MCP is an emerging open standard that lets AI assistants communicate with external systems (databases, APIs, file repositories) through a unified, discoverable interface. The advantage of MCP-style integration is that it brings consistency and reliability to how the LLM fetches information. Instead of hard-coding a separate API call for each internal tool (which leads to brittle, siloed solutions), we define a small set of MCP-like endpoints that the assistant can invoke for data. For our prototype, we will likely implement a few critical MCP endpoints such as: “`code_search(query)`” to search the codebase, “`doc_lookup(topic)`” to fetch relevant internal documentation pages, or “`ci_status(build_id)`” to get CI pipeline results. The assistant’s prompt will include a description of these available actions (following the MCP schema), enabling the LLM to request an action when needed (e.g., ask for a code search if the query is about locating a function). A thin orchestrator will intercept such requests, call the appropriate internal API or database, and return the results to the LLM in a structured format. By using an MCP-style approach, each tool integration is standardized and logged, making the system easier to extend and debug. This design also aligns with the goal of on-prem deployment: we can add connectors for any internal system (issue tracker, Slack logs, etc.) without exposing data externally, since the MCP servers run locally. Ultimately, GraphRAG and MCP complement each other: GraphRAG provides deep structured knowledge of the domain, while MCP provides live access to up-to-date information and actions. Together, they ensure our assistant has both the stored knowledge and the real-time data access to be maximally useful.

4 Implementation Plan

The project will be implemented in incremental stages, focusing on delivering a working prototype by the end of two months. The plan is outlined below, including what is in scope and out of scope:

In Scope:

- *LLM Selection and Setup:* Choose an appropriate open-source LLM (e.g., llama3.2:8b:instruct) that can run on available hardware. We will set up the model for efficient inference (using GPU acceleration and quantization if needed). No training from scratch will be done, but we may perform light fine-tuning or prompt tuning if necessary for our domain (e.g., few-shot exemplars for question-answering format).
- *Data Ingestion and Indexing:* Gather a representative sample of the organization’s internal knowledge for the prototype. This includes pulling a subset of source code repositories, exporting relevant Confluence/wiki pages, and collecting recent Q&A or tickets. We’ll then process this data to create two forms of indexes: (a) a vector index of text chunks (using embeddings to enable semantic search), and (b) a knowledge graph index

(entities and relations extracted from code and text). For the vector store, we can use an open-source library (FAISS or similar). For the graph, we will utilize GraphRAG tooling or build a simple custom extractor for key relations (e.g., function call graph, service dependency graph) to demonstrate the concept.

- *GraphRAG Integration:* Implement the GraphRAG-based retrieval pipeline. This involves using the indexed graph to improve query handling. We might use an existing GraphRAG library or API if available (Microsoft’s implementation is open-sourced), or write a simplified version due to time constraints. The goal is to show that for certain complex questions, the system can fetch not just isolated text, but a connected subgraph of information. We will also generate concise summaries of nodes or clusters in the graph (possibly by prompting the LLM or using existing summary fields in docs) to include in the context.
- *MCP Interface Implementation:* Develop a minimal MCP-style interface layer. Given the limited time, we will focus on one or two key tool integrations to prove the concept. For instance, we will implement an MCP server for code search by wrapping an existing code search tool or simple grep over the repository index. Likewise, a documentation lookup MCP server can be created to fetch pages by title or perform keyword search in docs. We will define a JSON-based request/response format for these actions as per MCP guidelines. On the client side (the LLM application), we’ll incorporate a mechanism for the model to request these actions. If using an open-source framework (like LangChain or LlamaIndex), we can plug into their tool invocation mechanisms; otherwise, we’ll write a simple loop that detects special tokens/commands in the LLM output indicating a tool use.
- *User Interface and Testing:* Build a simple interface for end-users (developers) to interact with the assistant. This could be a command-line chat session or a lightweight web app. The interface will display the conversation and allow asking questions. It will also show sourced citations for any facts or code provided (e.g., file names or document titles from which an answer was drawn). We will test the system on the example queries from the motivation section and additional realistic scenarios collected from colleagues. Feedback will be used to refine the retrieval prompts, add any needed prompt tuning, and address errors (e.g., if the assistant hallucinates without using the tools, we might adjust its prompt to encourage tool use).

Out of Scope:

- Training new LLMs or substantially extending model size is out of scope. We rely on existing pre-trained models due to time constraints.

- Full enterprise-scale integration of all company data. The project will handle a limited, curated dataset to demonstrate functionality, rather than ingesting the entire corporate codebase or all tools.
- Robust security/hardening of the system for production use. We will implement basic access controls (since the system runs internally, this is less of an issue), but things like authentication, auditing, or extensive safety filters are not the focus in this prototype.
- Comprehensive UI/UX design. The user interface will be functional but minimal. Integration into IDEs or chat platforms is a possible future direction, but the course project will prioritize the backend capabilities over polish.

By the end of implementation, we expect to have a self-contained system where a developer can ask questions and the assistant will fetch from the local knowledge base and respond accurately. The plan above ensures we tackle each technical component needed (LLM, retrieval, graph, tool interface) in manageable pieces.

5 Timeline (8 Weeks)

1. **Week 1: Requirement Analysis and Setup.** Define key use-cases and example queries. Set up development environment and obtain the chosen LLM model. Install necessary libraries for vector databases, NLP, and graph processing. Verify we can run the model locally (e.g., test some basic prompts).
2. **Week 2: Data Collection & Indexing.** Collect a pilot dataset from internal sources (perhaps one representative project’s code and associated docs). Implement the text chunking and embedding pipeline; build the vector index. Begin extracting a simple knowledge graph (could use static analysis for code or NLP entity extraction for text). Confirm that we can query the vector index for relevant snippets given a sample question.
3. **Week 3: Basic RAG Pipeline.** Integrate the LLM with the vector retrieval: build a routine that given a question, retrieves top- k relevant chunks and constructs an augmented prompt for the LLM. Develop a baseline Q&A prompt template. Test the end-to-end flow on straightforward questions (e.g., “Where is function X defined?”) and iterate on prompt formatting to get clear, cited answers. This week focuses on getting a minimal RAG-based QA working.
4. **Week 4: GraphRAG Enhancement.** Incorporate the knowledge graph into the retrieval process. Use the graph to handle a multi-hop query scenario: for example, implement logic such that if a question mentions two related entities, the system finds their connection via the graph. If using an existing GraphRAG toolkit, configure it on our data and compare

its results to the baseline. Fine-tune how graph-based context is merged with text snippets for the prompt. By end of week, we aim to demonstrate one or two queries where GraphRAG clearly improves the relevance of provided context (e.g., answering a question that spans multiple files or services).

5. **Week 5: MCP Interface Development.** Implement the backend for one or two MCP-style tools. For instance, set up a simple HTTP or IPC service that can perform code search in the repository. Define the JSON schema for requests (query string, etc.) and responses (file paths, code excerpts). Also implement a dummy tool if needed for another data source (like returning a wiki page snippet for a given title). Ensure these services run locally and can be called programmatically.
6. **Week 6: Tool Integration with LLM.** Modify the LLM interaction loop to allow tool usage. This may involve a special prompting technique (e.g., prefix the conversation with a system message enumerating available tools and how to call them, as per MCP). Alternatively, use an agent framework that supports tool plugins. Test the assistant's ability to invoke the code search tool when appropriate: e.g., ask a question about a function name not in its context, see if it emits a tool call, fetch results, and then continues with an answer. Iterate until the process is reliable for basic cases. By this week, the assistant should be able to automatically fetch unseen information during a conversation via our MCP endpoints.
7. **Week 7: Testing, Tuning, and Refinement.** Perform extensive testing with a set of diverse questions: API usage, debugging errors, finding who last modified a piece of code, etc. Identify failure modes. Refine the prompt (or even fine-tune the model on a few QA pairs) to reduce hallucinations and improve factuality. If the assistant is not using the tools when it should, adjust the prompt instructions to encourage that (possibly by showing an example in the prompt). Work on answer formatting: ensure the assistant cites filenames or document titles from which information was drawn, for transparency. Also measure latency for retrieval and response to gauge performance, and optimize if needed (e.g., limiting context size).
8. **Week 8: Final Evaluation and Documentation.** Define a small evaluation set of questions that reflect the project's goals and measure success (for instance, compare answers from our system vs. answers from ChatGPT given no internal data). Summarize the results (qualitatively, since this is a prototype). Compile documentation on system design, how to deploy it, and usage instructions. Prepare a short demonstration (recorded or live) to show to the course instructors, illustrating the assistant answering a few example queries. Buffer time is included in this week for any last-minute fixes or improvements discovered during final testing.

This timeline is aggressive but achievable, as each week builds upon the previous. Regular progress check-ins and adjustments will ensure the project stays on track. The deliverables will include the running prototype and a report detailing the design and findings.

6 Expected Outcome

By the end of the project, we expect to have an operational prototype of an on-premises LLM-based assistant that can handle questions requiring context from the organization’s internal software engineering knowledge. Concretely, the system will be able to ingest a query from a developer and produce a helpful answer that references the relevant internal sources (code files, documentation pages, etc.). For example, if asked, "What does the function `getUserPermissions` do in our auth service and has it changed recently?", the assistant might return an answer along the lines of:

“The function ‘`getUserPermissions`’ in the `AuthService` is defined in ‘`auth-service/Utils/Permissions.java`’ and is responsible for aggregating a user’s roles into a consolidated permission set. It calls the ‘`RoleService`’ API to fetch role definitionsreference to code. As of version 2.3 (updated 2 months ago), it was modified to include a cache layer for performancereference to Git history.”

This answer (hypothetical) would demonstrate the assistant’s ability to pull in specific code context and repository history information that a generic model like ChatGPT would not possess. We will measure success qualitatively by the correctness and relevance of answers to a set of test queries that were previously unanswerable without internal knowledge. We also expect the system to illustrate effective use of GraphRAG (by correctly linking information from multiple sources when needed) and MCP-style tool usage (by dynamically retrieving fresh information, such as the latest build status or recent commit messages, upon request).

The final outcome is not a production-ready system but a proof-of-concept that validates our approach. It will highlight how an LLM, when augmented with structured retrieval and tool interfaces, can serve as a context-aware assistant for software engineers. In summary, we anticipate demonstrating a working assistant that runs securely on company infrastructure and significantly outperforms vanilla LLMs on organization-specific Q&A tasks. This could lay the groundwork for future expansion, such as scaling to more data, integrating with developer workflows (IDE plugins), or applying the approach to other domains where private data integration with LLMs is needed.

References

1. Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*. **NeurIPS 2020**. (Introduction of GPT-3, a 175B-parameter model)

demonstrating strong NLP capabilities).

2. Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374. (OpenAI Codex paper; GPT-3 fine-tuned on GitHub code, powering GitHub Copilot).
3. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. **NeurIPS 2020**. (Introduces RAG, combining an LLM with an external document retriever to utilize non-training data).
4. Han, H., et al. (2025). *Retrieval-Augmented Generation with Graphs (GraphRAG)*. arXiv:2501.00309. (Survey of GraphRAG techniques; using knowledge graphs to improve RAG reasoning on private datasets).
5. Anthropic (2024). *Introducing the Model Context Protocol (MCP)*. (Announcement of MCP standard for connecting AI assistants to internal data/tools in a structured way).